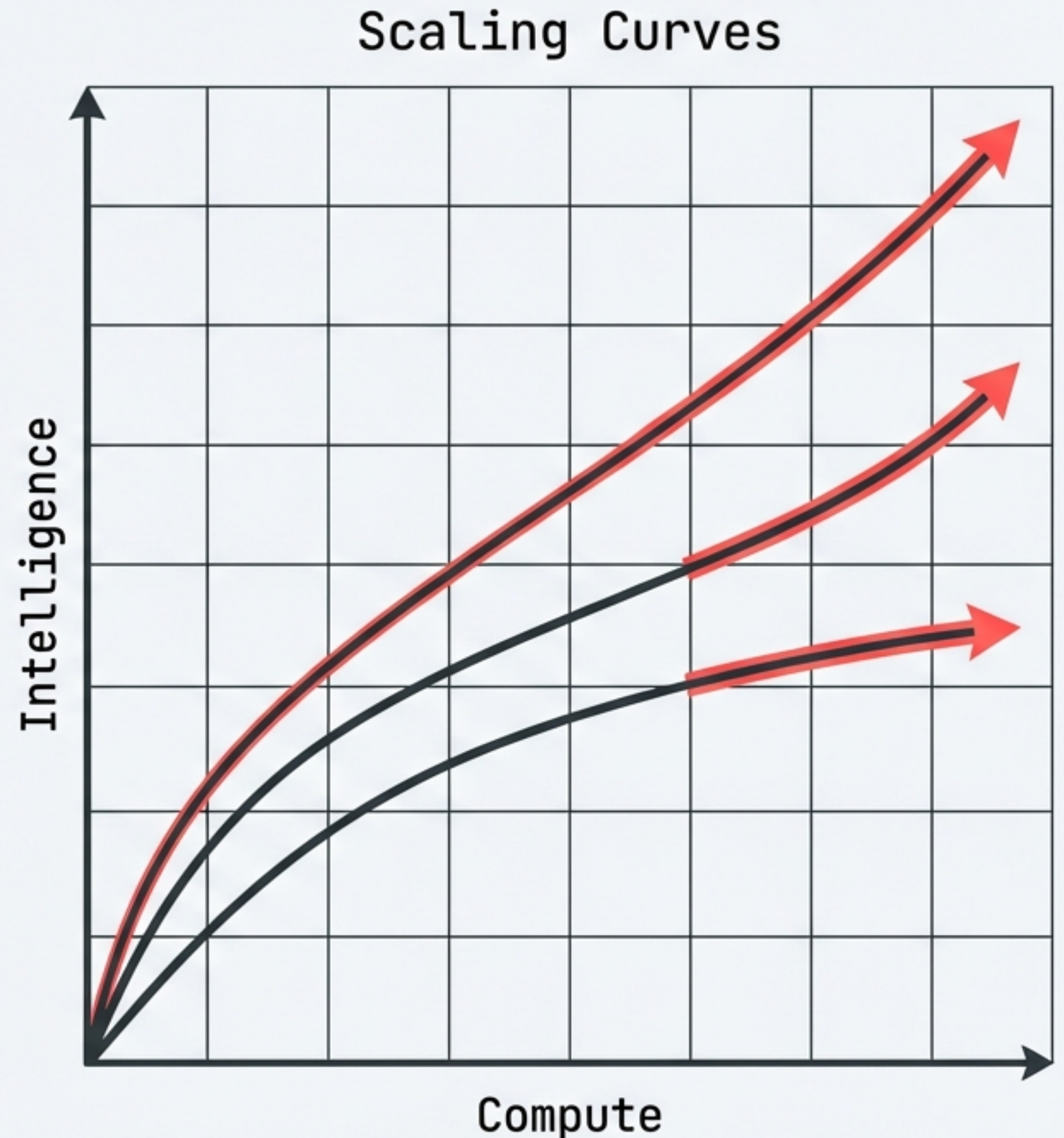


สำรวจโลก Reinforcement Learning สำหรับ Large Reasoning Models (LRMs)

จากความสามารถทางภาษา
สู่ก้าวกระโดดแห่งการให้เหตุผล

รายงานสรุปเชิงลึกด้านเทคนิค 2025: ครอบครัว OpenAI
o1, DeepSeek-R1 และสถาปัตยกรรมแห่งอนาคต

อ้างอิงจากงานวิจัย: A Survey of Reinforcement Learning for Large Reasoning Models (2025)



กฎการขยายขนาดแบบใหม่: จาก Data Scaling สู่ สู่ Compute Scaling

1. **Test-time Compute:** โมเดลอย่าง OpenAI o1 Test-time 9 และ DeepSeek-R1 แสดงให้เห็นว่าประสิทธิภาพจะเพิ่มขึ้นตามระยะเวลาที่ใช้ในการคิด (Thinking Time) และปริมาณ Compute ที่ใช้ระหว่างการอนุมาน (Inference)
2. LRMs (Large Reasoning Models): คือวิวัฒนาการใหม่ของ LLMs ที่ไม่ได้ทำหน้าที่แค่จับคู่รูปแบบภาษา (Pattern Matchers) แต่สามารถวางแผน ไตร่ตรอง และแก้ไขข้อผิดพลาดด้วยตนเอง

เรากำลังเปลี่ยนผ่านจากยุคที่
'ป้อนข้อมูลให้เยอะขึ้น' (Pre-training)
สู่ยุคที่ 'ให้เวลาคิดให้นานขึ้น'
(Test-time Compute)



จุดเปลี่ยนทางเทคนิค: จาก RLHF เพื่อความปลอดภัย สู่ RLVR เพื่อความฉลาด

The Old Paradigm (RLHF/DPO)



- Goal: การจัดระเบียบพฤติกรรมมนุษย์ (Human Alignment) - ช่วยเหลือ, ซื่อสัตย์, ไม่เป็นอันตราย (3H)
- Feedback: ความพึงพอใจของมนุษย์ (Subjective)
- Outcome: แชทบอทที่สุภาพและปลอดภัย

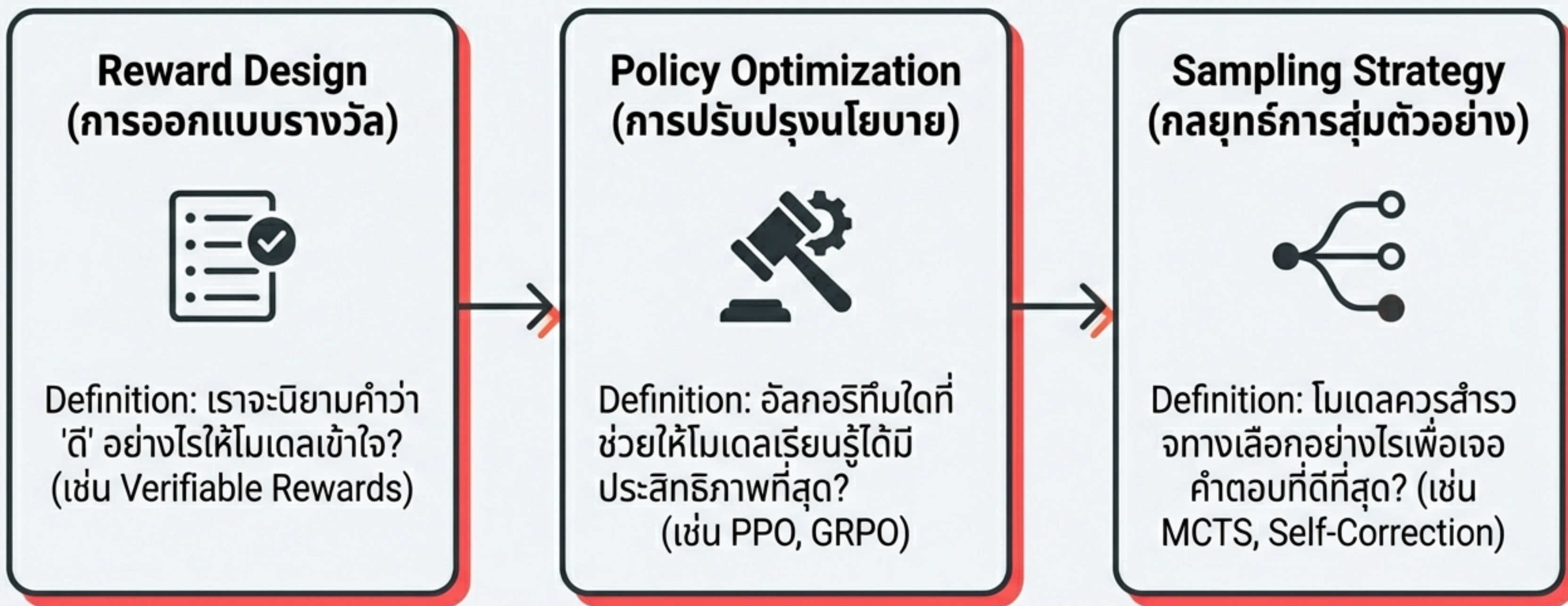
The New Paradigm (RLVR)



- Goal: การเพิ่มขีดความสามารถในการให้เหตุผล (Reasoning Capabilities)
- Feedback: กฎเกณฑ์ที่ตรวจสอบได้จริง (Objective) เช่น ผลลัพธ์ทางคณิตศาสตร์ หรือการรันโค้ด
- Outcome: โมเดลอัจฉริยะที่แก้โจทย์ซับซ้อนได้ (เช่น DeepSeek-R1)

องค์ประกอบพื้นฐานของระบบ Reinforced Reasoning

การสร้าง LRM ต้องอาศัย 3 กลไกหลักทำงานประสานกัน (Based on Section 3 Taxonomy)



Reward Design: พลังแห่งรางวัลที่ตรวจสอบได้ (Verifiable Rewards)

Verifier's Law: Helvetica Now Display Prompt, JetBrains Mono English
Verifier's Law: หากเราสามารถตรวจสอบความถูกต้องของคำตอบได้โดยอัตโนมัติ
เราจะสามารถขยายขีดความสามารถของปัญญาประดิษฐ์ได้อย่างไร้ขีดจำกัด

Rule-based Rewards



Math:

ตรวจสอบคำตอบสุดท้ายว่า
ตรงกับเฉลยหรือไม่
(Answer Correctness)



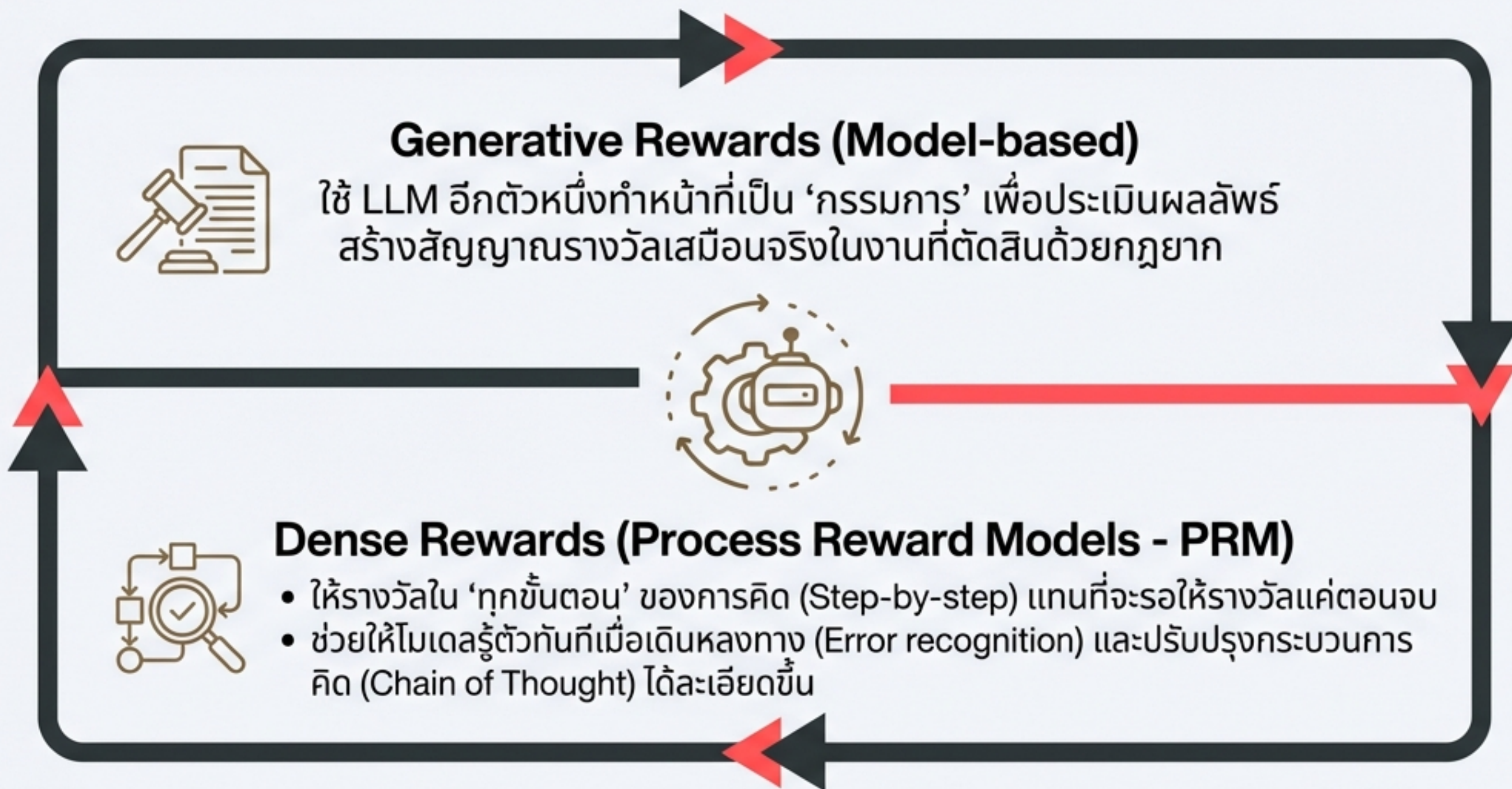
Code:

Code ตรวจสอบว่าโค้ดสามารถ
Compile ผ่านและผ่าน Unit
Tests ทั้งหมดหรือไม่

- **Why it works:** ป้องกันปัญหา “Reward Hacking”
และสร้างสัญญาณการเรียนรู้ที่แม่นยำกว่าการใช้คนประเมิน

นอกเหนือจากการตรวจสอบ: Generative และ Dense Rewards

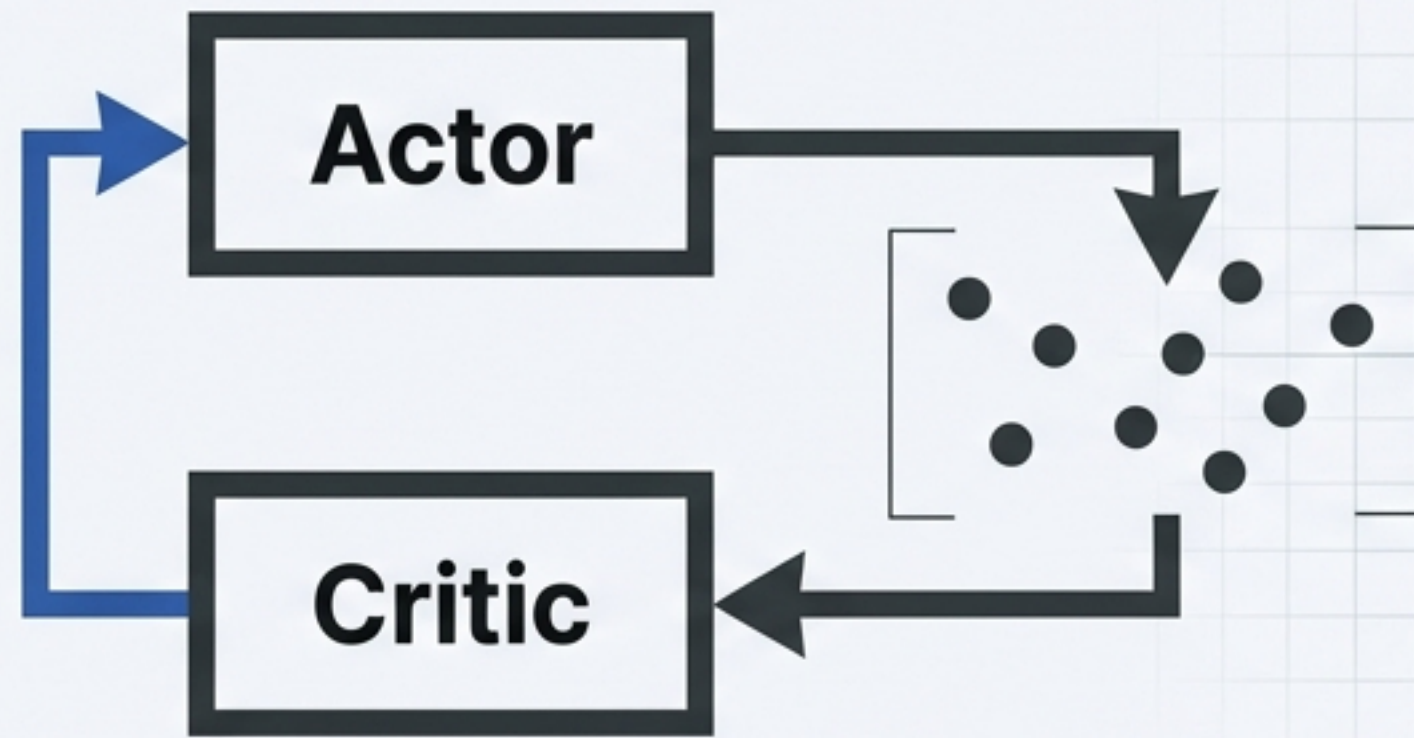
Problem: สำหรับงานที่ไม่มีคำตอบตายตัว (เช่น การเขียนบทความ) หรือโจทย์ที่ซับซ้อนมาก เราต้องใช้เทคนิคขั้นสูง:



Policy Optimization: วัฒนาการจาก PPO สู่ GRPO

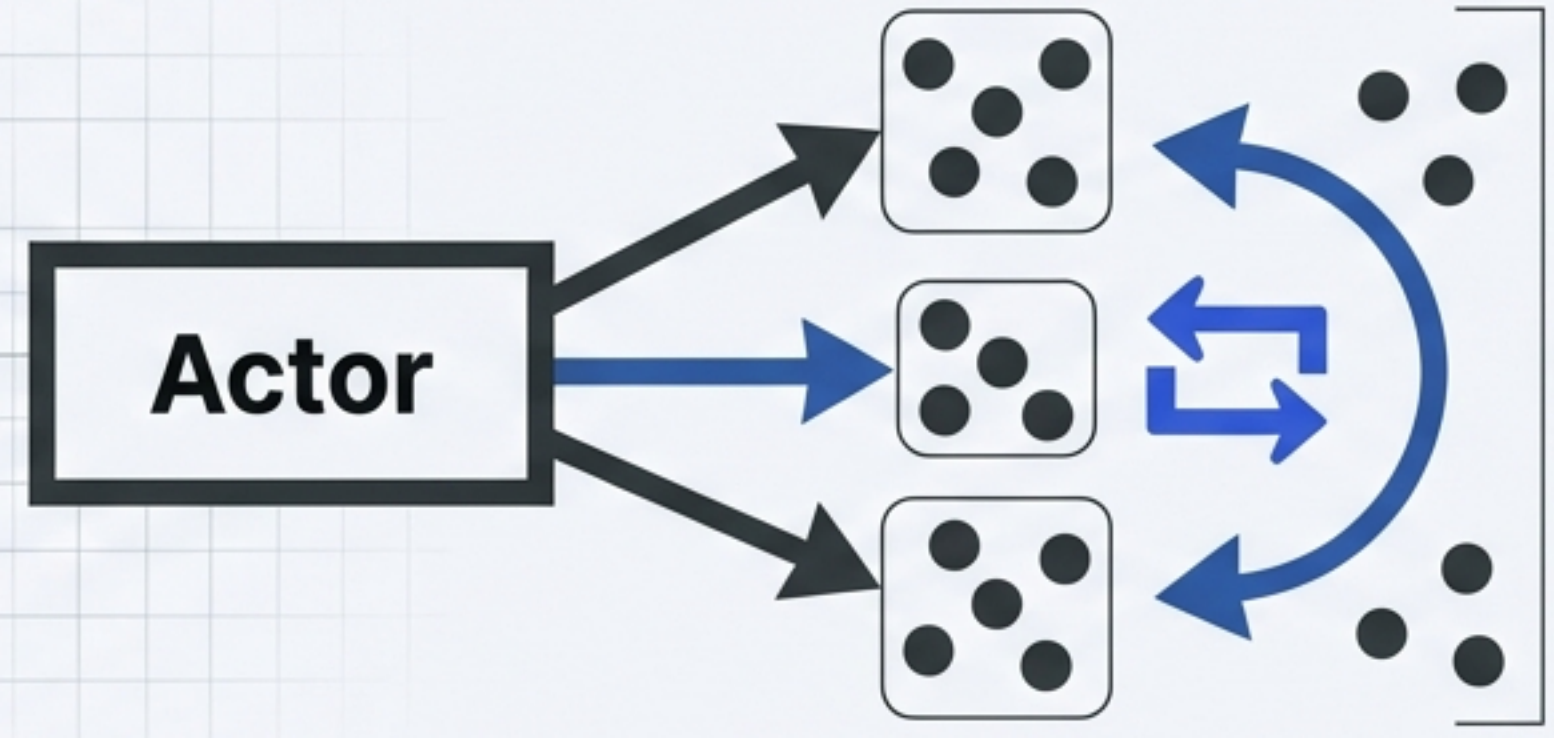
การปรับปรุงสมองของโมเดลให้ฉลาดขึ้นโดยไม่เปลืองทรัพยากร

Traditional: Critic-based (PPO)



- ต้องใช้โมเดล 'Critic' แยกต่างหากเพื่อประเมินค่า (Value Function)
- ข้อเสีย: ใช้หน่วยความจำมหาศาล (Memory-heavy) ทำงานช้า

State-of-the-Art: Critic-free (GRPO)



- ใช้ใน DeepSeek-R1: ไม่ต้องมี Critic Model แยก
- ใช้วิธี Group Relative Policy Optimization: สุ่มคำตอบมาเป็นกลุ่ม (Group) แล้วเปรียบเทียบกันเองภายในกลุ่มเพื่อหาคำตอบที่ดีที่สุด
- ข้อดี: ลดการใช้ทรัพยากร และรองรับการขยายขนาด (Scale) ได้ดีกว่ามาก

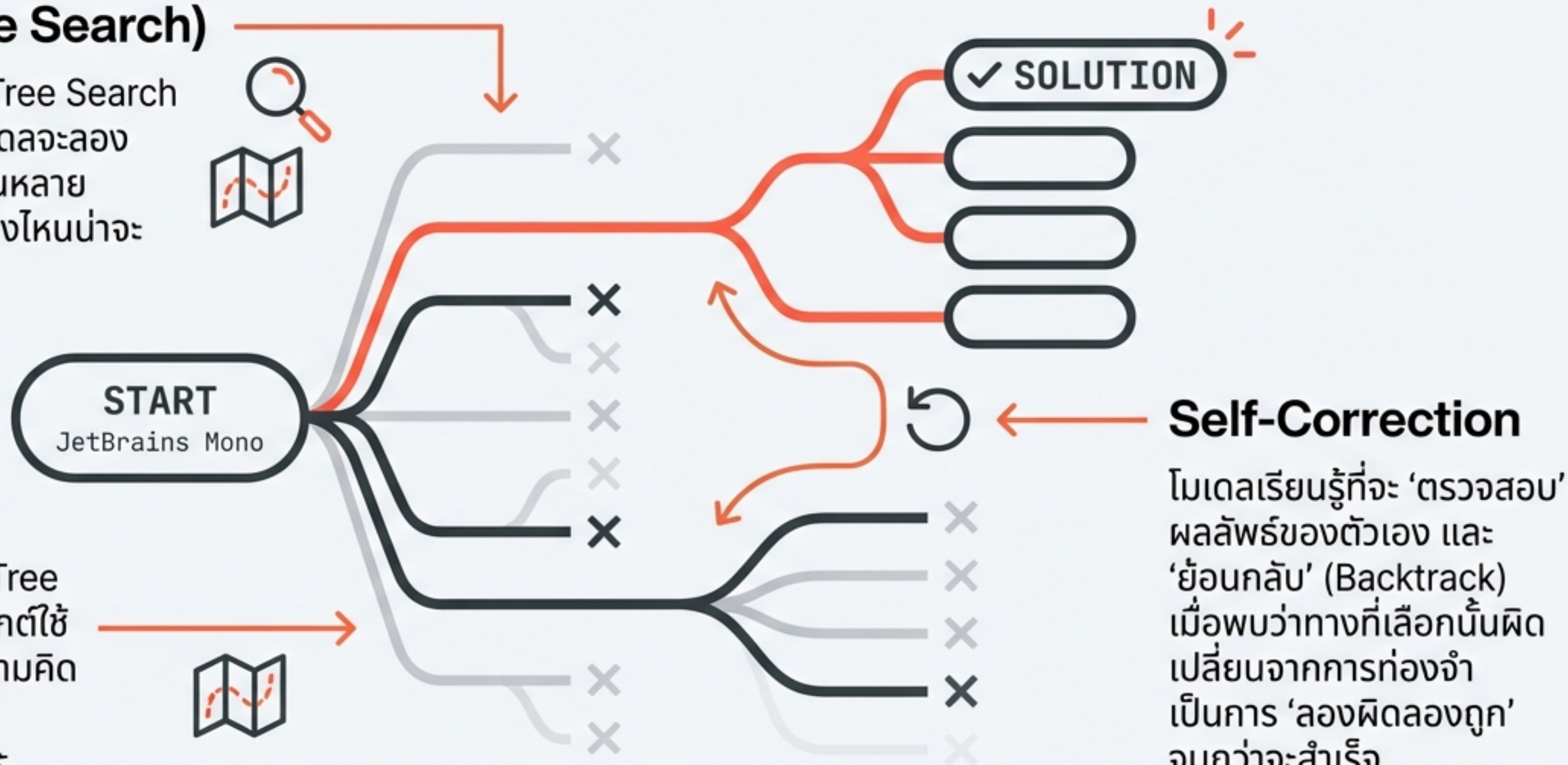
Sampling Strategy: การสำรวจและแก้ไขตนเอง

การค้นหาคำตอบที่ดีที่สุดไม่ได้มาจากการเดาครั้งเดียว แต่เกิดจากการสำรวจหลายเส้นทาง

Exploration (Tree Search)

นำแนวคิด Monte Carlo Tree Search (MCTS) มาประยุกต์ใช้ โมเดลจะลองแตกแขนงความคิดออกเป็นหลายทางเลือก แล้วประเมินว่าทางไหนน่าจะไปสู่คำตอบที่ถูกต้อง

นำแนวคิด Monte Carlo Tree Search (MCTS) มาประยุกต์ใช้ โมเดลจะลองแตกแขนงความคิดออกเป็นหลายทางเลือก แล้วประเมินว่าทางไหนน่าจะไปสู่คำตอบที่ถูกต้อง



ความท้าทายหลัก: การท่องจำ (Memorization) vs การเข้าใจจริง (Generalization)

โมเดลกำลัง 'คิด' จริงๆ หรือแค่จำแพทเทิร์นการตอบ?

SFT (Supervised Fine-Tuning)



มักนำไปสู่การเลียนแบบข้อมูลฝึกสอน (Mimicry) ซึ่งเสี่ยงต่อการท่องจำ

RL (Reinforcement Learning)



บังคับให้โมเดลค้นหาวิธีแก้ปัญหาด้วยตัวเอง (Discovery) ซึ่งนำไปสู่การประยุกต์ใช้กับโจทย์ที่ไม่เคยเห็นมาก่อน (Generalization) ได้ดีกว่า

The Goal: สร้างโมเดลที่สามารถแก้โจทย์ 'Out-of-distribution' หรือโจทย์รูปแบบใหม่ๆ ที่ไม่อยู่ในฐานข้อมูลได้

ทรัพยากรสำหรับการฝึกฝน: ข้อมูลและสภาพแวดล้อม Helvetica Now Display

Static Corpus (ข้อมูลนิ่ง)

โจทย์คณิตศาสตร์ (Math Problems) และคลังโค้ด (Code Repositories)
สำหรับปูพื้นฐาน



Dynamic Environments (สภาพแวดล้อมพลวัต)

- Compilers & Interpreters: เครื่องมือรันโค้ดที่ให้ผลลัพธ์ทันที (Pass/Fail)
- Interactive Sandboxes: พื้นที่ให้ Agent ลองใช้เครื่องมือ (Tools) และรับ Feedback
- Feedback Loops: ระบบที่ให้รางวัลและบทลงโทษตามผลลัพธ์จริง ไม่ใช่แค่ความเหมือนของข้อความ



Application Spotlight: การเขียนโปรแกรม (Coding Tasks)

Why it leads: Coding เป็นพื้นที่ที่ RL เติบโตได้ดีที่สุดเพราะมี 'Ground Truth' ที่ชัดเจน

> terminal ~ self-debugging

```
def calculate_factorial(n):...  
    return n * calculate_factorial(n-1) —————>
```

↳ **Error:** RecursionError: maximum recursion depth exceeded ❗

Execution Feedback (Self-Debugging)

```
def calculate_factorial(n):...  
    result = 1...  
    for i in range(1, n+1):... —————> Corrected  
        result *= i...           JetBrains Mono  
    return result
```

→ **Success:** calculate_factorial(5) -> 120 ✅



Unit Tests as Rewards: ใช้ผลการรัน Test Case เป็นรางวัลที่ตรวจสอบได้ทันที



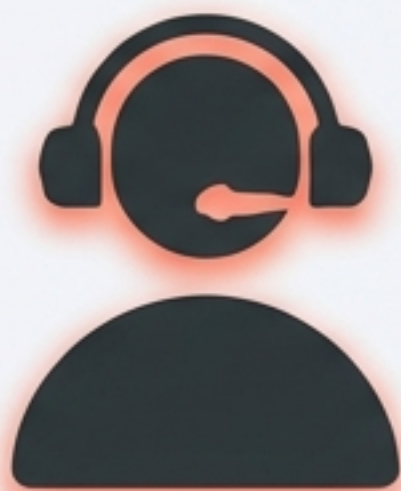
Self-Debugging: โมเดลสามารถอ่าน Error Message และแก้ไขโค้ดของตัวเองซ้ำๆ จนกว่าจะทำงานได้ (Execution Feedback)



Contests: รองรับโจทย์ระดับการแข่งขัน (Competitive Programming) เช่น LeetCode หรือ Codeforces

Application Spotlight: Agents และระบบ Multimodal

Agentic Tasks (การทำงานแบบตัวแทน)



- การวางแผนระยะยาว (Long-horizon planning) เช่น การจองตั๋วเครื่องบิน หรือการสืบค้นข้อมูลบนเว็บ
- การใช้เครื่องมือ (Tool Use) เพื่อแก้ปัญหาที่ซับซ้อนหลายขั้นตอน

Multimodal Reasoning (การให้เหตุผลแบบหลายรูปแบบ)



- การนำ RL มาใช้กับข้อมูลภาพ (Vision)
- เช่น การทำความเข้าใจชาร์ตที่ซับซ้อน (Chart Understanding) หรือ การแก้ไขข้อผิดพลาดจากรูปภาพ
- โมเดลอย่าง QVQ และ Gemini แสดงให้เห็นศักยภาพในการ 'มองเห็นและคิดวิเคราะห์' ไปพร้อมกัน

ภูมิทัศน์ของ Frontier Models ในปัจจุบัน

การแข่งขันระหว่างโมเดลปิด (Proprietary) และโมเดลเปิด (Open Source)

Proprietary Leaders

- **OpenAI:** o1, o3 (เน้น Chain of Thought และ RLVR เต็มรูปแบบ)
- **Anthropic:** Claude 3.7 (เน้น Coding และ Agentic capabilities)

Open Source Challengers

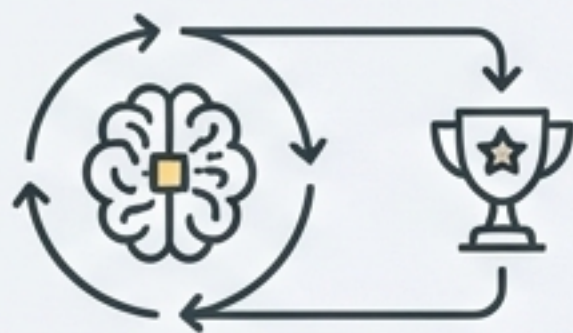
- **DeepSeek:** DeepSeek-R1 (ผู้บุกเบิก GRPO และ Pure RL)
- **Qwen:** QwQ และ Qwen3 (ประสิทธิภาพสูงในงานตรรกะ)
- **Skywork:** Skywork-OR1 (เน้นกระบวนการฝึกฝนที่มีประสิทธิภาพ)

อนาคตของ RL สำหรับ LLMs: มุ่งสู่ Artificial SuperIntelligence (ASI)



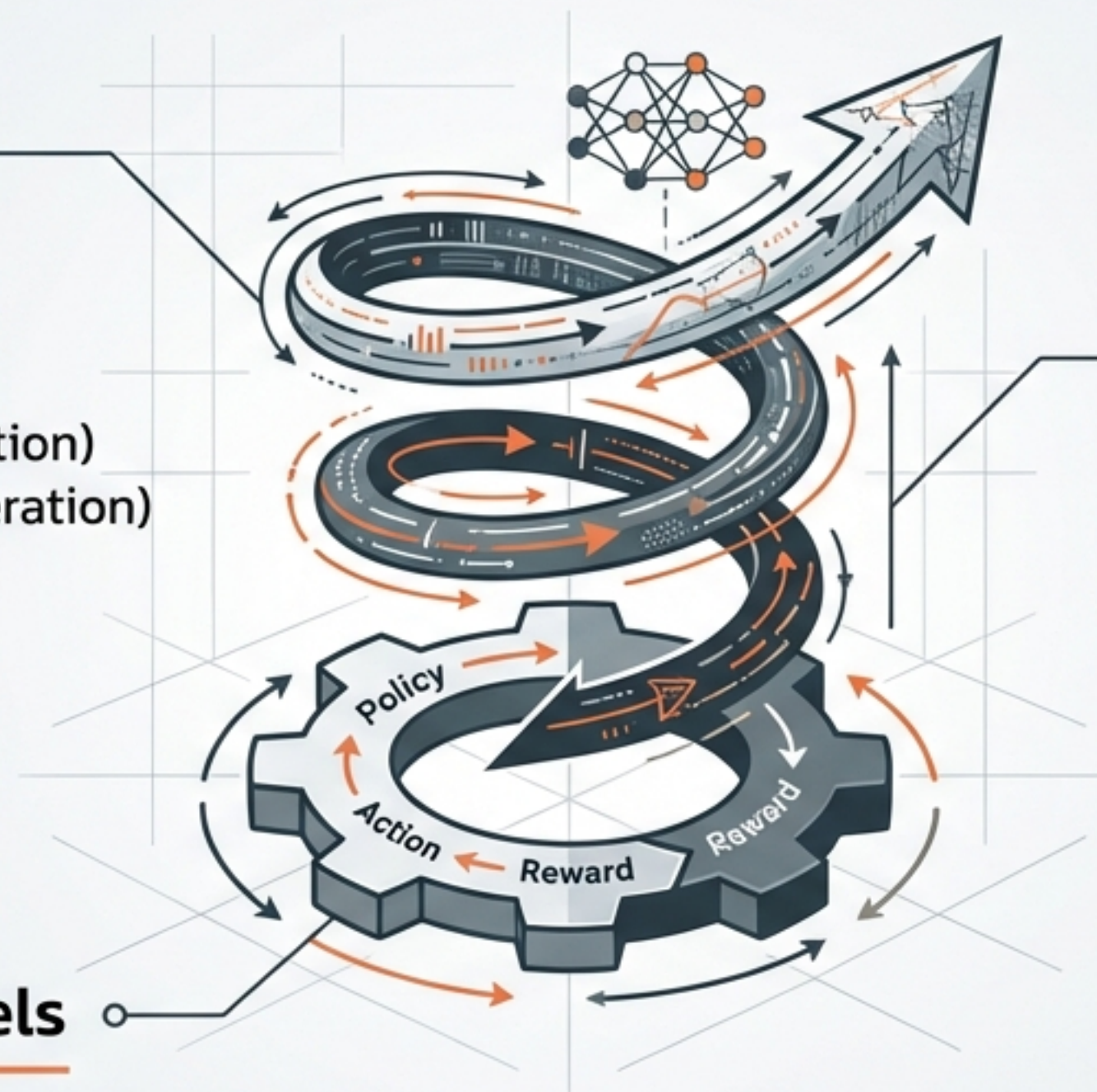
System 1 → System 2

เปลี่ยนการตอบสนองที่รวดเร็ว (Intuition) ให้เป็นการคิดวิเคราะห์ที่ลึกซึ้ง (Deliberation) และมีประสิทธิภาพมากขึ้น



Self-Evolving Models

โมเดลที่สามารถออกแบบรางวัล (Reward Function) ให้ตัวเองได้ เพื่อเรียนรู้ในเรื่องที่มนุษย์ไม่สามารถตรวจสอบได้อีกต่อไป



Latent Space Reasoning

การคิดในรูปแบบที่ไม่ใช่ภาษา (Non-linguistic thought) เพื่อความรวดเร็วและเป็นนามธรรมมากขึ้น

บทสรุป: ยุคแห่งการแก้ปัญหา (The Era of Problem Solving)



- 1. **RL คือเครื่องยนต์:** Reinforcement Learning ไม่ใช่แค่ส่วนเสริม แต่เป็นหัวใจหลักในการสร้างความฉลาด (Intelligence)
- 2. **Verification คือกุญแจ:** ความสามารถในการตรวจสอบความถูกต้อง (Verifiable Rewards) คือสิ่งที่ทำให้เราขยายขนาด (Scale) ได้
- 3. **The Shift:** เรากำลังก้าวข้ามจาก AI ที่ “เรียนรู้เพื่อสนทนา” (Learning to Chat) ไปสู่ AI ที่ “เรียนรู้เพื่อแก้ปัญหา” (Learning to Solve Problems)